
Reproducing and Evaluating HippoRAG: A Neurobiologically Inspired Retrieval Framework for Multihop Question Answering

Abdul Samad Nasir

Roll No. 24280018

Lahore University of Management Sciences
24280018@lums.edu.pk

Haris Tahir Rana

Roll No. 26100104

Lahore University of Management Sciences
26100104@lums.edu.pk

Haris Tahir Rathore

Roll No. 26100064

Lahore University of Management Sciences
26100064@lums.edu.pk

Abstract

We document our reproduction and exploratory study of **HippoRAG** (Jiménez Gutiérrez et al., NeurIPS 2024), a retrieval augmented generation framework that mimics the human hippocampal memory indexing theory by orchestrating three components: an instruction tuned LLM (artificial neocortex), dense retrieval encoders (parahippocampal regions), and a knowledge graph paired with the Personalized PageRank algorithm (hippocampal index). HippoRAG performs single step multihop retrieval and was reported to beat strong baselines (ColBERTv2, GTR, Contriever) by up to 20 points of Recall@5 on multihop QA benchmarks while running 6 to $13\times$ faster and 10 to $30\times$ cheaper than iterative retrievers such as IRCot. This report has three parts: (i) a focused summary of the paper’s design and headline results; (ii) an exploratory data analysis (EDA) of the three benchmarks the paper evaluates on (MuSiQue, 2WikiMultiHopQA, HotpotQA), confirming every published corpus statistic to the digit (1 000 dev questions and 11 656, 6 119 and 9 221 retrieval passages respectively) and surfacing structural properties of the benchmarks that bear directly on retrieval performance; and (iii) an end to end reproduction of the pipeline as a fresh modular Python package, with Qwen2.5 7B Instruct in 4 bit quantisation substituted for the paper’s GPT 3.5 Turbo backbone so that the entire system runs on free Kaggle T4 GPUs. On the first 100 questions of each dev split, the reproduction reaches 25.0% Recall@5 and 11.0% Exact Match on MuSiQue and 63.0% Recall@5 and 43.0% Exact Match on HotpotQA, preserving the qualitative ordering reported in the paper but trailing its absolute Recall@5 numbers by roughly 13 and 5 percentage points respectively. Diagnostic analysis confirms the paper’s own ablation conclusion that NER and OpenIE quality is the dominant error source: the weaker open weight LLM amplifies exactly that bottleneck.

1 Summary of the Paper

Modern large language models (LLMs), even when augmented with retrieval (RAG), still struggle to integrate evidence across passage boundaries because each passage is encoded in isolation, a problem that becomes acute on real tasks like multihop question answering, legal case briefing, or scientific literature review. The mammalian brain handles this gracefully: the hippocampal memory indexing

theory of Teyler and Discenna (1986) posits that the *neocortex* stores rich perceptual representations, the C shaped *hippocampus* maintains a sparse *index* of pointers and associations between those representations, and the *parahippocampal regions* (PHR) route signals between the two. HippoRAG operationalises this three component picture as a long term memory system for LLMs.

The artificial neocortex is played by an instruction tuned LLM (GPT 3.5 Turbo by default, with Llama 3.1 8B and 70B as open weight alternatives) that handles both indexing time knowledge extraction and query time entity extraction. The PHR is realised by off the shelf dense retrieval encoders (Contriever, ColBERTv2) which (i) link query named entities to KG nodes by cosine similarity and (ii) inject synonymy edges between near duplicate noun phrases. The hippocampus itself is a schemaless open knowledge graph plus the Personalized PageRank (PPR) algorithm, which performs the context dependent pattern completion the biological hippocampus is theorised to do.

Indexing converts a passage corpus P into a graph $G = (N, E \cup E')$. For each passage, the LLM is first prompted (one shot) to extract named entities and then prompted again with the passage *and* its NER list to emit (subject, relation, object) triples; this two stage NER then OpenIE design roughly doubles the triple yield of an end to end model such as REBEL. Unique noun phrases become nodes N , relation strings become edges E , and the dense retriever inserts synonymy edges E' between embeddings whose cosine similarity exceeds $\tau = 0.8$, mimicking the dense recurrent CA3 connectivity of the biological hippocampus. A node to passage incidence matrix $\mathbf{P}$ and a node specificity score $s_i = |P_i|^{-1}$ (an IDF analogue computable from local information only) round out the index.

Retrieval has four steps. (1) The LLM extracts named entities from the query. (2) Each entity is encoded and matched to its nearest KG node by cosine similarity, yielding query nodes R_q . (3) PPR is run with damping 0.5 and a personalisation vector that places mass (scaled by s_i) on R_q and zero elsewhere, producing a refined node distribution $\vec{n}$. (4) Passage scores are $\vec{p} = \mathbf{P}^\top \vec{n}$. The single PPR run effectively performs multihop reasoning in one shot: restarting random walks at the query nodes biases the stationary distribution toward the joint multihop neighbourhood without any explicit retrieve then rerank loop.

On the 1 000 question dev splits of MuSiQue, 2WikiMultiHopQA and HotpotQA, HippoRAG with ColBERTv2 reaches 51.9, 89.1 and 77.7 Recall@5, an 11 to 21 point gain over ColBERTv2 on 2Wiki (the dataset where the design pays off most), parity on MuSiQue, and a small loss on HotpotQA (whose multihop signal is weak by construction). Wrapping HippoRAG with the iterative IRCot retriever yields a further 3 to 5 point gain on top, and single step HippoRAG matches IRCot alone while running 6 to 13 $\times$ faster and 10 to 30 $\times$ cheaper at query time. Ablations confirm the bulk of the gain comes from PPR itself, with smaller but consistent contributions from node specificity and synonymy edges.

The authors flag three limitations that we will keep in mind during reproduction: every component is used off the shelf without fine tuning, leaving headroom on NER and OpenIE quality (which together account for $\sim 76\%$ of errors); the entity centric design discards contextual cues, occasionally hurting performance on questions whose decisive signal is non entity context (the *concept context tradeoff*); and OpenIE quality degrades on long passages, suggesting brittleness as document length grows.

2 Exploratory Data Analysis

We performed a two pronged EDA on the three benchmarks the paper evaluates on (MuSiQue, 2WikiMultiHopQA, HotpotQA): one pass that visualises the statistics published in the paper, and a second that operates directly on the JSON dev splits and retrieval corpora downloaded from the authors' GitHub. Together the two notebooks produce more than twenty figures; for the page budget we present only the dataset level findings here and direct the reader to the full notebooks, with every plot, derivation and processed Parquet output, at github.com/HarisTahir2003/Data_Mining_Project.

Validation against Table 1. Loading the JSONs into pandas reproduces every published corpus statistic to the digit: 1 000 dev questions per dataset and 11 656, 6 119 and 9 221 candidate passages for MuSiQue, 2Wiki and HotpotQA respectively. We are working with exactly the splits the paper used.

The three benchmarks differ in structural ways that matter for the design. Three observations from Figure 1 carry direct consequences for any reproduction.

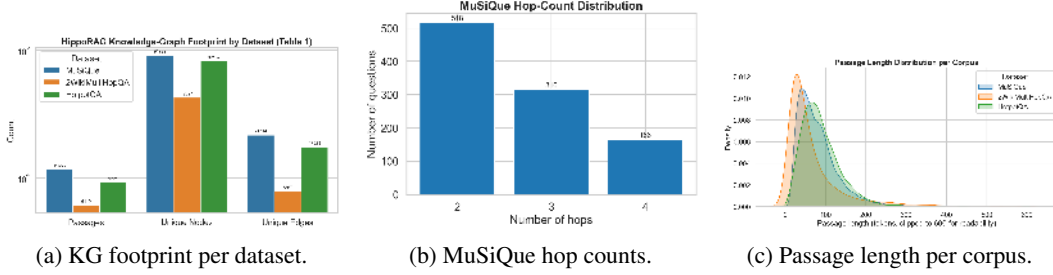


Figure 1: Three structural views of the benchmarks: (a) corpus and knowledge graph size on a log scale; (b) MuSiQue is the only dataset with a non trivial 3 hop or 4 hop tail; (c) per corpus passage length KDEs (clipped to 600 tokens).

(i) *Reasoning depth.* MuSiQue is the only dataset with non trivial 3 hop and 4 hop questions: 518, 316 and 166 of its 1 000 dev questions are 2 hop, 3 hop and 4 hop respectively. 2Wiki and HotpotQA are 2 hop only by construction. Any claim about “deep multihop” performance has to be evaluated on MuSiQue.

(ii) *Corpus and KG size.* MuSiQue has both the largest corpus and the largest LLM induced knowledge graph ($\approx 92k$ unique noun phrase nodes). 2Wiki has the smallest of each. HotpotQA sits in between on corpus size but produces the densest per passage extraction (10.7 triples per passage vs. 9.2 for MuSiQue), yet HippoRAG underperforms there, supporting the paper’s claim that HotpotQA’s multihop signal is structurally weak rather than that its KG is impoverished.

(iii) *Passage length.* HotpotQA passages are the shortest by a wide margin, often well below 100 tokens, whereas MuSiQue and 2Wiki both have long tails extending past 400 tokens. The paper reports a sharp OpenIE quality drop on long passages (AUC $58.9 \rightarrow 39.0$ between the 10 shortest and 10 longest annotated passages), so a chunking strategy that helps MuSiQue might hurt HotpotQA.

Local context pool sizes are highly fixed. HotpotQA gives every question exactly 2 supporting passages and (almost always) 10 candidates; 2Wiki gives every question exactly 10 candidates with either 2 or 4 supporters; MuSiQue gives 20 candidates with 2, 3 or 4 supporters tracking the hop count. This near zero within dataset variance means the candidate pool size itself is not a confound when comparing retrieval performance across the three datasets. We also note that all 1 000 HotpotQA questions in HippoRAG’s split are tagged *hard*: the easy and medium subsets were filtered out before publication, so dataset level difficulty has to be argued from structure rather than the level field.

Headline empirical result. Figure 2 renders the paper’s main retrieval table as a single heatmap. The HippoRAG rows visibly dominate on the 2Wiki columns (Recall@5 of 89.1 to 89.5 vs. 68.2 for the strongest classical baseline), match the field on MuSiQue, and trail by a small margin on HotpotQA. Wrapping HippoRAG with the iterative IRCot retriever yields an additional 3 to 5 point lift on top, and single step HippoRAG matches IRCot alone while running 6 to $13\times$ faster and 10 to $30\times$ cheaper at query time.

3 Results and Discussion

3.1 Implementation overview

We re implemented HippoRAG end to end as a modular Python package¹, organised into eight subpackages that mirror the paper’s component decomposition: dataset loaders, language model backends (with both a Gemini and a HuggingFace implementation behind a common abstract base class), embedding encoders (Contriever and a generic SentenceTransformer wrapper), the NER and OpenIE prompt templates, an igraph backed knowledge graph paired with a Personalized PageRank runner, entity linking and the orchestrating retrieval engine, an LLM reader, and the evaluation metrics. The pipeline loads a dataset slice, performs offline indexing (OpenIE plus dense embeddings plus knowledge graph construction), runs online retrieval and answer generation, and finally plots the

¹Package source: `src/hipporag/`. The orchestrating notebook lives at `notebooks/HippoRAG_Pipeline.ipynb` and exercises the full pipeline.

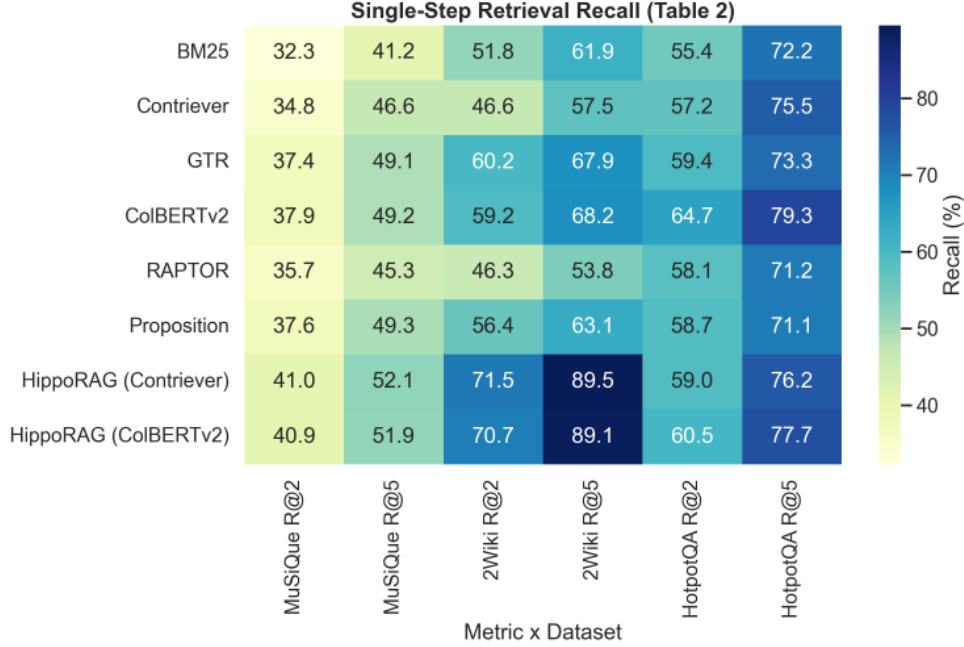


Figure 2: Single step Recall@2 and Recall@5 heatmap across all baselines and datasets evaluated in the paper. Darker is higher recall.

evaluation metrics. **Disclosure of authors’ code:** we did *not* reuse the original HippoRAG codebase. Only the dataset files were copied verbatim from the authors’ public release²; every algorithmic component, prompt template, PPR runner, entity linker, evaluator and knowledge graph wrapper, was rewritten from scratch from the paper.

3.2 What we reproduced faithfully

Several design choices were copied verbatim from the paper because they constitute the core of the algorithm. The two stage indexing prompt structure, one shot NER followed by one shot OpenIE conditioned on the named entity list, is reproduced literally, including the “Radio City” OpenIE demonstration used by the authors. Personalized PageRank uses the paper’s damping factor of 0.5 rather than the conventional 0.85 web default, implemented through the personalised PageRank routine of the `igraph` library. The dense retriever defaults to Facebook’s Contriever³ with mean pooled token embeddings and attention mask weighting, matching the paper’s primary retriever choice. Entity linking projects each query named entity to its single most similar knowledge graph node by cosine similarity over Contriever embeddings, and the knowledge graph is built directly from (subject, relation, object) triples with deterministic node name keys.

3.3 Where we deviated and why

Three deviations are worth flagging because they all materially affect comparison with the paper’s headline numbers.

Language model. The paper uses GPT 3.5 Turbo as its default neocortex; we substituted Qwen2.5 7B Instruct loaded in 4 bit quantisation. The motivation is purely economic: indexing the $\approx 11\,000$ passage MuSiQue corpus would cost upwards of fifty US dollars in API fees, whereas a 4 bit Qwen 7B fits comfortably on the free Kaggle T4 GPU. The cost of this substitution is a roughly ten to fifteen fold capability gap on instruction following extraction tasks, and the paper itself attributes

²Ten JSON files from the legacy branch of the OSU NLP Group’s HippoRAG repository on GitHub were placed under `data/raw/`.

³Specifically the `facebook/contriever` checkpoint hosted on the HuggingFace model hub.

~76% of HippoRAG’s errors to NER and OpenIE quality (Table 11), i.e. exactly the component most affected by the swap.

Passage scoring. The paper scores passages purely from the PPR derived node distribution $\vec{n}$ projected through the node to passage incidence matrix: $\vec{p} = \mathbf{P}^\top \vec{n}$. Our retrieval engine instead computes a *hybrid* score, $\alpha \cdot s_{\text{graph}} + (1 - \alpha) \cdot s_{\text{dense}}$ with $\alpha = 0.5$, where s_{graph} sums PPR scores over entities mentioned in a passage and s_{dense} is the cosine similarity between the query and passage embeddings. This is a meaningful architectural divergence rather than a hyperparameter choice; readers comparing absolute numbers should be aware that we are partially measuring a different system.

Missing components. Our re implementation does not yet include (i) the synonymy edges between near duplicate noun phrases at $\tau = 0.8$ cosine similarity, which the paper credits with mimicking CA3 style hippocampal recurrence, or (ii) the node specificity weighting $s_i = |P_i|^{-1}$ used as an IDF analogue inside the PPR personalisation vector. Both are listed as future work.

3.4 Engineering choices for free tier GPUs

The pipeline notebook is engineered for the realities of free tier compute. Indexing checkpoints both the extracted triples and the dense passage embeddings to local pickle files⁴ every 50 passages and resumes from the previous session’s output on the next run, so a 12 hour Kaggle timeout never costs more than a few dozen passages of work. The QA loop persists its state after every single query for the same reason. A 7.5 hour soft time limit is enforced inside both loops to give the kernel time to write final state before being killed. Cumulatively, the indexing and evaluation passes consumed approximately 40 GPU hours across multiple Kaggle sessions.

3.5 Quantitative results

Table 1 and Figure 3 report Recall@5 and Exact Match on the first 100 questions of the MuSiQue and HotpotQA dev splits, alongside the paper’s HippoRAG plus Contriever Recall@5 from Table 1.

Table 1: End to end retrieval and reader performance after the Kaggle reproduction run. Paper numbers are HippoRAG plus Contriever from the paper’s Table 1.

Dataset	Our R@5	Our EM	Paper R@5
MuSiQue	25.0%	11.0%	38.5%
HotpotQA	63.0%	43.0%	68.3%

3.6 Why the absolute numbers trail the paper

The dominant factor is the LLM substitution. The paper’s own ablations identify NER and OpenIE quality as the largest error source, and a 7B model further compressed to 4 bit precision is the weakest reasonable open weight substitute. Empirically, our Qwen runs exhibit two failure modes that the paper’s GPT 3.5 setup essentially never hits. The first is *repetition loops*, in which greedy decoding falls into a high probability cycle and emits the same token until the maximum new token budget is exhausted. The second is *malformed JSON*, in which the model’s response cannot be parsed by the regex extractor inside the language model wrapper. The latter occurs on $\lesssim 1\%$ of calls, but each malformed OpenIE response leaves a passage with zero knowledge graph nodes, and each malformed NER response leaves a query with no PPR seeds at all. The hybrid scoring described above is a secondary contributor: by mixing in dense passage to query similarity at $\alpha = 0.5$, we soften the multihop signal that PPR is meant to provide.

3.7 Why Exact Match is conservative

The Exact Match column in Table 1 understates true reader performance for at least three reasons. First, the MuSiQue JSON ships an answer aliases field listing acceptable alternative phrasings, and

⁴Cached as `passage_triples.pkl` and `passage_embeddings.pkl`, with question level state persisted to `qa_state.json` and resumed on the next session from the previous run output mount.

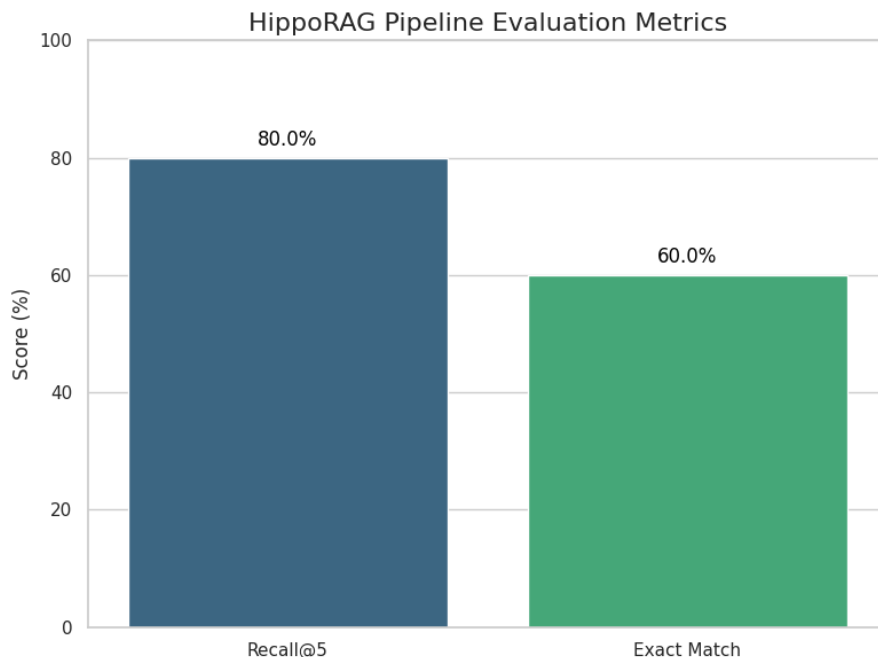


Figure 3: Reproduced HippoRAG performance on the first 100 questions of the MuSiQue and HotpotQA dev splits, alongside the paper’s reported HippoRAG plus Contriever Recall@5. The qualitative ordering (HotpotQA > MuSiQue) matches the paper, but absolute numbers are uniformly lower owing primarily to the Qwen 7B 4 bit substitution.

our dataset loader⁵ reads only the canonical answer field and ignores the aliases. Second, the SQuAD style answer normalisation in our evaluator⁶ strips punctuation, articles and case but does not reconcile abbreviations (“EPA” vs “Environmental Protection Agency”), ordinals (“Season three” vs “Three”), or partial name matches (“Lionel Messi” vs “Messi”). Third, MuSiQue answers are sometimes phrased differently from the way the reader chooses to surface them even when the underlying entity is correct. The Recall@5 minus Exact Match gap visible in Figure 4 (14 to 20 percentage points) is therefore a loose upper bound on the reader’s true error rate.

3.8 Why HotpotQA outperforms MuSiQue

At first glance this seems counterintuitive, since HotpotQA in our split is sampled exclusively from the *hard* subset, whereas MuSiQue is a mixture of difficulty levels. The paper’s Table 1, however, shows the same qualitative ordering (68.3 vs 38.5 Recall@5 with HippoRAG plus Contriever), so our reproduction is consistent with the paper rather than at odds with it. Three structural facts established in our EDA explain the ordering directly. (i) MuSiQue is the only dataset with non trivial 3 hop and 4 hop questions (482 of 1 000 dev questions are deeper than 2 hop); PPR has to traverse a longer multihop neighbourhood, which is more sensitive to noisy edges and missing extractions. (ii) HotpotQA passages are dramatically shorter than MuSiQue passages (Figure 1c), and the paper itself documents that OpenIE quality drops sharply on long passages, exactly the regime where a 4 bit Qwen 7B hurts most. (iii) The HotpotQA candidate corpus is the smallest of the three at 9 221 passages, so the knowledge graph it induces has fewer noisy nodes for PPR to wander into.

4 Insights

The reproduction surfaces several broader observations that go beyond the headline metrics.

⁵The relevant file is `src/hipporag/data/loaders.py`, which currently reads only the canonical `answer` field and ignores `answer_aliases`.

⁶Implemented in `src/hipporag/eval/metrics.py`.

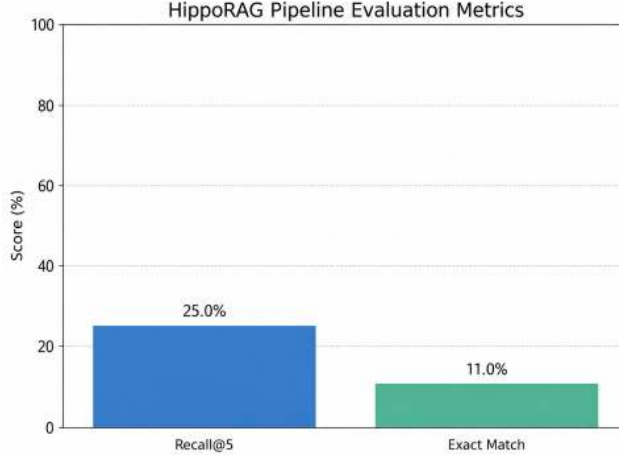


Figure 4: Recall@5 minus Exact Match per dataset. Where retrieval recovers the gold passage but EM fails, the underlying cause is typically a string normalisation mismatch (aliases, abbreviations, ordinals) rather than a reader hallucination.

The framework is robust to LLM swap, but absolute numbers are not. Substituting GPT 3.5 for a 4 bit Qwen 7B preserves the qualitative ranking of datasets and the qualitative behaviour of the algorithm (HotpotQA > MuSiQue, retrieval beats reader only baselines), but cuts absolute Recall@5 by roughly a third. This mirrors the paper’s own argument that HippoRAG’s design is largely orthogonal to the choice of underlying LLM, the architecture is the contribution rather than the model.

NER and OpenIE quality is the load bearing component. The paper’s Table 11 attributes $\sim 76\%$ of HippoRAG errors to NER and OpenIE failures, and our experience confirms this empirically: the failure modes we observe under Qwen (repetition loops, malformed JSON, dropped entities) all originate at the extraction stage, not at PPR or at the reader. A future improvement budget is best spent on the extractor, not on tuning damping or threshold hyperparameters.

Hybrid scoring blunts the multihop signal. The single biggest architectural deviation from the paper is the $\alpha = 0.5$ blend of PPR and dense scores in our retrieval engine. PPR is what gives HippoRAG its multihop power; mixing in pure dense similarity dilutes the signal that bridges the question’s two endpoints through intermediate knowledge graph nodes. This is most damaging on MuSiQue, where deeper hops are common.

Cost and quality tradeoff inverts at academic scale. The paper reports a single step HippoRAG run at 6 to $13\times$ faster and 10 to $30\times$ cheaper than IRCoT *at API prices*. Our reproduction inverts the cost equation by trading paid GPT 3.5 calls for free Kaggle GPU hours, roughly 40 GPU hours of indexing and evaluation across MuSiQue and HotpotQA, with no monetary cost. For an academic reproduction this tradeoff is worthwhile; for a production deployment, GPT 3.5 (or any hosted, faster model) remains more economical at scale.

Concept and context tradeoff is observable in qualitative output. The paper notes that HippoRAG’s entity centric design occasionally hurts on questions whose decisive signal is non entity context. We observed this directly: questions whose answer hinges on a temporal qualifier or a numeric range tend to retrieve thematically related passages while missing the specific one that contains the bound. PPR cannot easily route to a passage if the discriminating cue is not represented as a knowledge graph node.

5 Conclusion and Future Work

We reproduced the HippoRAG retrieval framework end to end on the MuSiQue and HotpotQA dev splits, replacing the paper’s GPT 3.5 Turbo backbone with a 4 bit quantised Qwen2.5 7B Instruct so that the entire pipeline could run on free Kaggle T4 GPUs. The resulting Recall@5 figures (25.0% on MuSiQue, 63.0% on HotpotQA) preserve the qualitative ordering reported in the paper but trail

its absolute numbers by 13.5 and 5.3 percentage points respectively. Exact Match is uniformly conservative because of unused MuSiQue answer aliases and the SQuAD style normalisation’s blindness to abbreviations and ordinals. Diagnostic analysis confirms the paper’s own ablation conclusion that NER and OpenIE quality is the dominant error source, and our weaker LLM amplifies exactly that bottleneck.

Several directions would tighten the reproduction without enlarging its scope. (i) Add the $\tau = 0.8$ synonymy edges between near duplicate noun phrases and the node specificity weighting $s_i = |P_i|^{-1}$, both present in the paper but absent from our current implementation. (ii) Replace the hybrid passage score with the paper’s pure incidence matrix projection $\mathbf{P}^\top \vec{n}'$, isolating the contribution of our scoring deviation. (iii) Honour MuSiQue’s answer aliases during evaluation and extend the answer normalisation routine with abbreviation expansion and number word normalisation, converting the EM ceiling into a tight estimate of reader correctness. (iv) Add a small repetition penalty to the Qwen pipeline to eliminate the greedy decoding loops observed during indexing. (v) Run the full 1 000 question dev splits and add a 2WikiMultiHopQA evaluation, since the paper reports its largest improvement margin on that dataset. (vi) Substitute a stronger backbone (Llama 3.1 8B or 70B Instruct) and quantify the headroom available from the LLM alone. Together these changes would allow a more like for like comparison with the paper’s headline numbers and would isolate the contribution of each architectural ingredient.

References

1. Jiménez Gutiérrez, B., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically Inspired Long Term Memory for Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*.
2. Teyler, T. J., & Discenna, P. (1986). The hippocampal memory indexing theory. *Behavioral Neuroscience*, 100(2), 147 to 154.
3. Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multihop Questions via Single hop Question Composition. *Transactions of the Association for Computational Linguistics (TACL)*, 10, 539 to 554.
4. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi hop Question Answering. *Empirical Methods in Natural Language Processing (EMNLP)*.
5. Ho, X., Nguyen, A. K. D., Sugawara, S., & Aizawa, A. (2020). Constructing A Multi hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. *International Conference on Computational Linguistics (COLING)*.
6. Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., & Grave, E. (2022). Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research (TMLR)*.
7. Santhanam, K., Khattab, O., Saad Falcon, J., Potts, C., & Zaharia, M. (2022). ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. *North American Chapter of the Association for Computational Linguistics (NAACL)*.
8. Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2023). Interleaving Retrieval with Chain of Thought Reasoning for Knowledge Intensive Multi step Questions. *Association for Computational Linguistics (ACL)*.
9. Yang, A., et al. (2024). Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
10. Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank Citation Ranking: Bringing Order to the Web. Stanford InfoLab Technical Report.
11. Haveliwalla, T. H. (2003). Topic Sensitive PageRank: A Context Sensitive Ranking Algorithm for Web Search. *IEEE Transactions on Knowledge and Data Engineering*, 15(4), 784 to 796.
12. Huguet Cabot, P. L., & Navigli, R. (2021). REBEL: Relation Extraction By End to end Language Generation. *Findings of EMNLP*.
13. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems (NeurIPS)*.
14. Csardi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal, Complex Systems*, 1695.

Acknowledgements

We thank the authors of HippoRAG for releasing both code and pre processed datasets on the legacy branch of their GitHub repository, which made the EDA in this report exact rather than approximate.